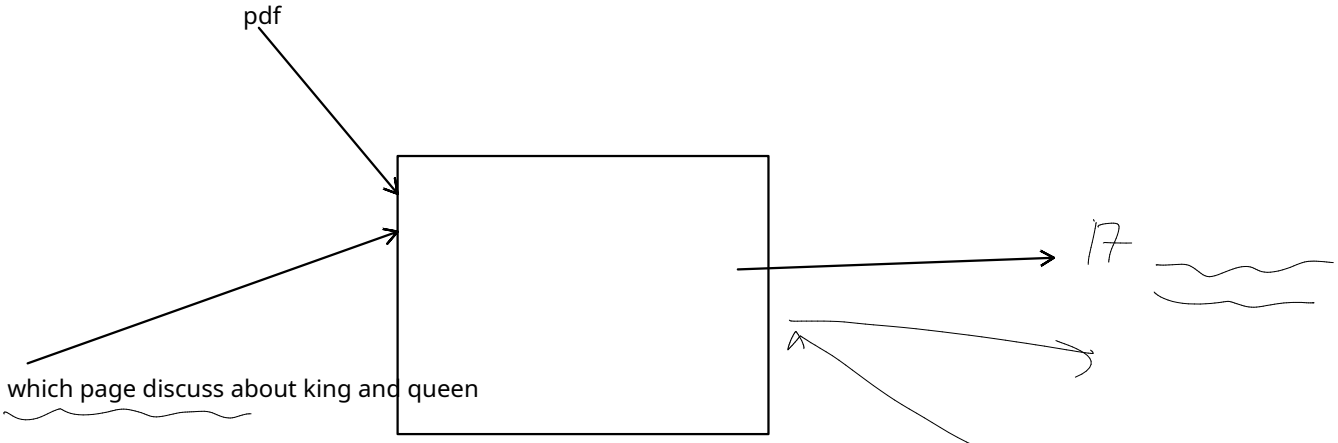
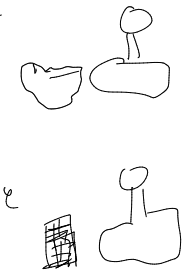


parse → chunk → each chunk → [ ]

Boy eats apple.

apple
↳ fruit
↳ Phone



He has apple phone.

apple → Binary encoder → 0 1 0 1 0 1 . . . → apple

apple → Numbers contains Meaning.
↳ [10, 5, 6, 7, ----]

1) How
2) Why Numbers useful?

dimp

Abstraction

300 → 1000

	isAlive	How Big?	How Influencial?	isItFood
King	[1	6	99	2]
Queen	[1	5	75	2]
Apple	[0	1	0	98]
Elephant	[1	15	20	5]
Order	[0	1	99	0]

Vector Embedding

→ 1D Dim Number } data

Order
- King = x
- Queen = y
- App = z
- Ele = p

x >

How llm works? ----- > it predict the next word

A order by King
↓
[- - - - -]

1000 Users
↓
500 query/day

PDF files ==> Atomic Habit ==> 200 to 5000 pages

You want to build a q&a chatbot, where user will ask quesiton from the book, and you have to mention, based on book?

GB file

Solution:

UserQuery+PDF ----> LLM ----> output ----> user

1-2 para 1000 paras → 1.5 K para → hallucinatio

÷ 100 day → 100

10 para 1 RS

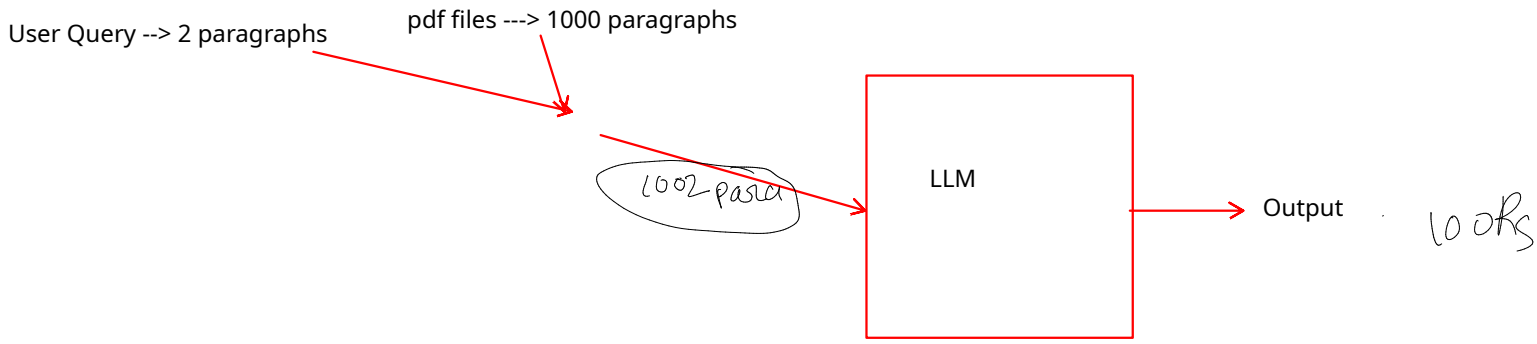
Problem:

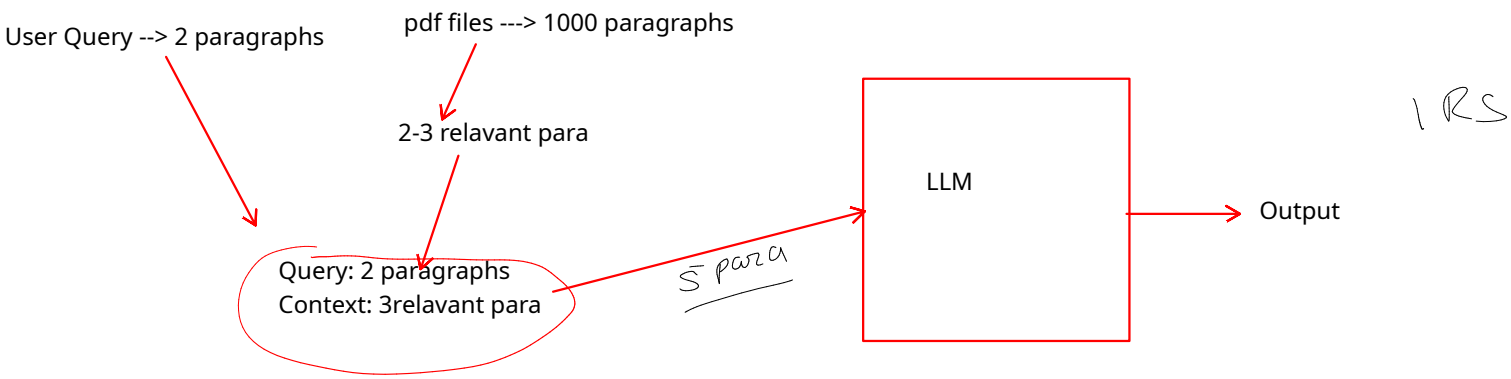
For each query ---> 1000paragraph pdf is sent

{context length Exceeded in future

Solution:

For each query ---> send the relevant 2-3 paragraphs





1---> Pdf ---> text

pdf file ---> pymupdf python library ---> ["page1 contet", ..., "last page content"]

1_pdf_read.py U × Book.pdf U

2_Embeddings > 1_pdf_read.py > doc

```python
import pymupdf

doc = pymupdf.open("Book.pdf")

print(len(doc)) # 285


print('-'*30)
print(doc[7].get_text())
print('-'*30)

# can we ["page1 data", "page2 text",.....]
# [doc[0], doc[1], doc[2], ...., doc[284]]

all_pages_array = [ page.get_text() for page in doc ]

print(all_pages_array[7])
```

2) text_array ----> embeding array

page1text ----> [4,14,5,,2,4,2,4....]

page2text ----> [5,24,f,v,,sd,,sd....]

page3text ----> [4,14,5,,2,4,2,4....]

embeding Model

free small → run on System

Local → Model →

1_pdf_read.py U 2_embeding.py U × Book.pdf U

2_Embeddings > 2_embeding.py > ...

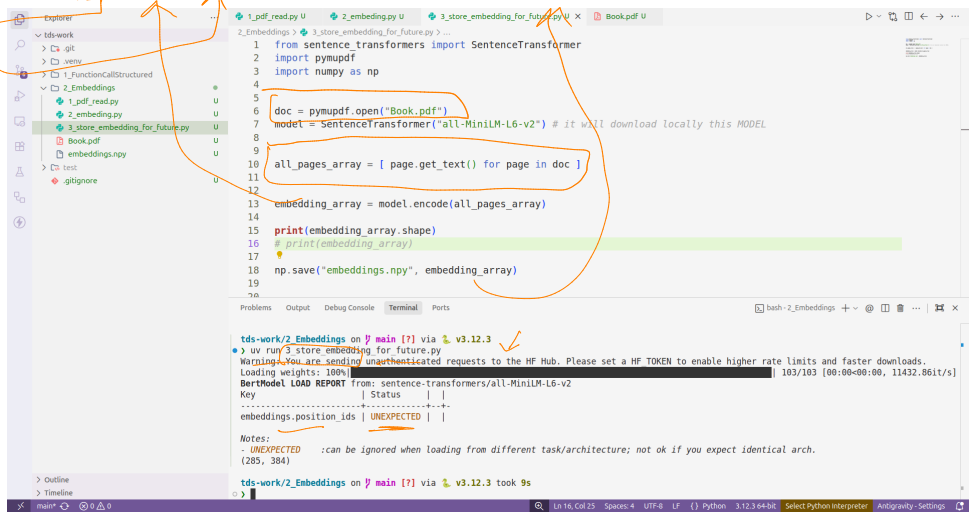
```python
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2") # it will download locally this MODEL


all_pages_array = ["page 1 data", "introduction"]

array = model.encode(all_pages_array)

print(array.shape)
```

(2, 286)

col = 286

page1 data	2	3	1	10	. .	.	. .
introduction	1	5	10	0	1	2	. .

3) pdf ----> ["page1 text", ...] ----> [[1,3,4...], [1,4,5,2,...],.....] ------> embedings.npy



4) user query + there relevant page

embeding.npy

pdf ----> [

page 1 embeddngs
page2 embedding
page3 embedding
....
....
]

285

[user Embedin]

that 2page Content

Retrieving data

2 page

IP

Prompt

LLM

```
prompt = f"""
    You will anser based on user query, and based on context only, qute the lines also
    User Query: {user_query}


    Context:
    ---------Page{two_index[0]}----------
    {doc[two_index[0]].get_text()}
    ---------Page{doc[two_index[1]].get_text()}----------
    {doc[two_index[1]]}

    """

print(prompt)


from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-4.1-nano",
    input=prompt
)

print(response.output_text)
```

there are many vector database, langchain

FIrst

---> creating embeding
---> np.dot to each embedng by QUERY VECTOR
----> SORT
-----> index ---> doc page no or chunk no.
---> chunk no.

Time

Vector db → 5 operatio

abstract

use qm

285

285
5000

dot

Sort 285

285 x 384

1000 OPT

1 Lakhs